IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Limpiar datos sin hacer trampas

Duplicados, tipos, fechas, categorías, imposibles y nulos

Sprint 6 · S06_02 · 55 min en clase (ejercicios 1, 2, 3 y 5)

MÉTODO

Nunca limpies a ciegas

1 · Mide

¿A cuántas filas afecta el defecto?

2 · Decide

Qué hacer con él, y **anota por qué**.

3 · Escribe

El código que lo corrige.

4 · Comprueba

Vuelve a medir: ¿ha mejorado?

```
crudo = pd.read_csv(dato("reservas_sucio.csv"))   # el original: no se toca
d1 = crudo.copy()                                 # paso 1: trabajas en una copia
d2 = d1.copy()                                    # paso 2: otra copia nueva
```

Como en la consulta médica: primero la analítica, después el tratamiento. Con una copia por paso ves **cuánto cambia cada decisión** y puedes volver atrás.

DIAGNÓSTICO

Mide antes de corregir

Resultado de **resumen_calidad(tabla)** con 6 reservas. **Nulo** = hueco (dato que falta). La última columna la añadimos nosotros.

columna	tipo	nulos	pct_nulos	distintos	así se ven
noches	int64	0	0.00	5	3 · 2 · 4 · 1 · 5 · 2
adultos	float64	1	16.67	3	2.0 · NaN · 3.0 · 2.0 …
canal	object	0	0.00	5	Directo · directo · Agencia · DIRECTO …
precio_noche	object	1	16.67	4	72,50 · 88 · 95.10 · NaN …

precio_noche es object

object es texto: un precio no debería serlo. Aquí lo estropea la **coma**.

adultos es float64

Un entero no admite huecos: con **un nulo** la columna pasa a decimal.

canal: 5 distintos

Solo hay **2 canales**: están mal escritos.

DUPLICADOS · EJERCICIO 1

La misma reserva, dos veces

id_reserva	noches	duplicated()
301	2	False
302	5	False
302	5	True
303	3	False
304	4	False

```
tabla.duplicated().sum()          # 1
d1 = tabla.drop_duplicates()      # índice 0, 1, 3, 4
d1 = d1.reset_index(drop=True)    # índice 0, 1, 2, 3
```

Por qué quitarlo

La reserva **pesa el doble** y puede acabar en entrenamiento y en prueba a la vez.

Por qué reiniciar el índice

Tras quitar filas el número de fila queda con **huecos**; **reset_index** lo renumera.

TIPOS · EJERCICIO 2

Números escritos como texto

Texto original	Tras convertir	Qué ha pasado
72,50	72.5	La coma pasa a punto: ya es un número
88	88.0	Sin coma: se convierte igual
vacío (NaN)	NaN	Ya estaba vacío
n/d	NaN	**Nulo nuevo**: coerce lo deja vacío. ¡Cuéntalos antes y después!

```
p = pd.Series(["72,50", "88", np.nan, "n/d"])
p.isna().sum()                                  # 1 nulo antes
n = pd.to_numeric(p.astype(str).str.replace(",", "."), errors="coerce")
n.isna().sum()                                  # 2 nulos después
```

FECHAS · EJERCICIO 3

Dos formatos, dos lecturas

«03/04/2025»: ¿3 de abril o 4 de marzo? **No adivines: lee cada grupo con su formato.**

texto	iso (año-mes-día)	esp (día/mes/año)	iso.fillna(esp)
2025-03-14	2025-03-14	NaT = fecha vacía	2025-03-14
27/01/2025	NaT	2025-01-27	2025-01-27
03/04/2025	NaT	2025-04-03	2025-04-03

```
iso = pd.to_datetime(f, format="%Y-%m-%d", errors="coerce")
esp = pd.to_datetime(f, format="%d/%m/%Y", errors="coerce")
(iso.isna() & esp.isna()).sum()     # 0: ninguna sin leer
f_ok = iso.fillna(esp)              # fechas completas
```

CATEGORÍAS · EJERCICIO 4 (CASA)

Normalizar categorías

Valores originales	Cómo se arregla	Resultado
Directo · "directo " · DIRECTO	strip y lower (quita espacios, minúsculas)	directo (los 3)
web intermediaria	además, espacio por _	web_intermediaria
España · es · Portugal	diccionario con **map**	ES · ES · PT
Marruecos (no está en el diccionario)	**map** no lo conoce	**NaN**: nulo nuevo

```
canal2 = canal.str.strip().str.lower().str.replace(" ", "_")   # 6 categorías pasan a 3
pais2 = pais.str.strip().str.lower().map(equiv)                # equiv: dict de equivalencias
pais.isna().sum(), pais2.isna().sum()                          # (0, 1): ¡un nulo nuevo!
```

Es para casa. En clase, ejecuta la celda «Solución de referencia» para tener **d4** y seguir con el ejercicio 5.

IMPOSIBLES Y RAROS · EJERCICIO 5

Imposible no es lo mismo que raro

Valor encontrado	¿Qué es?	Qué hago
Precio de −30 € por noche	**Imposible**	Pasarlo a nulo
0 noches de estancia	**Imposible**	Pasarlo a nulo
Antelación de −6 días	**Imposible**	Recalcularla con las fechas
Suite a 420 € cuando el resto ronda 70 €	**Raro, pero posible**	Investigar; no borrar

Imposible: error seguro, se corrige. Raro: un outlier (valor atípico), se investiga.

Para localizar sospechosos, la regla del **IQR** marca lo que cae fuera de **[Q1 − 1,5·IQR, Q3 + 1,5·IQR]**: con 58, 62, 65, 68, 70, 74, 77 y 420, marca solo el 420. Marcar no es condenar.

REGLAS DE NEGOCIO · EJERCICIO 5

Convertir los imposibles en nulo

Regla de ejemplo: el precio por noche debe estar **entre 30 y 500 €**.

```
p = pd.Series([64.0, -30.0, 78.5, 5000.0, 91.0, 12.0])
ok = p.between(30, 500)                    # True, False, True, False, True, False
limpio = p.where(ok)                       # 64.0, NaN, 78.5, NaN, 91.0, NaN
limpio.isna().sum() - p.isna().sum()       # 3 valores a nulo
```

between

¿Está en el rango? **Incluye** los dos extremos.

where

Conserva lo que cumple; **el resto pasa a nulo**.

¿Por qué no borrar la fila?

El resto de la fila sigue siendo válido: se tacha la casilla mala, no se tira el formulario.

COHERENCIA ENTRE COLUMNAS

Cruza columnas: mandan las fechas

fecha_reserva	fecha_llegada	antelacion_dias	calculada
2025-05-02	2025-05-12	10	10
2025-05-10	2025-06-09	25	30
2025-06-01	2025-06-07	−6	6

```
calc = (r["fecha_llegada"] - r["fecha_reserva"]).dt.days   # 10, 30, 6
(calc != r["antelacion_dias"]).sum()                       # 2 no cuadran
r["antelacion_dias"] = calc                                # las fechas mandan
```

Si no cuadra, la fuente de verdad son las fechas: se recalcula toda la columna.

NULOS

Un hueco: ¿olvido o respuesta?

Falta al azar

Pocos huecos: **imputar**, es decir, rellenar con la mediana (números) o la moda (texto).

Falta por algo

Un país vacío puede ser información: categoría **desconocido** o columna que avisa.

Falta casi todo

Si la columna está vacía en su mayoría, valora **quitarla**.

```
precios = pd.Series([55, None, 70, 400])
precios.mean()        # 175.0 (lo arrastra el 400)
precios.median()      # 70.0 (el valor del medio)
```

Aquí solo contamos y documentamos: imputar se hace dentro de un pipeline (S06_03).

MINI-RETO · LIMPIAR_RESERVAS (CASA)

Todo en una función reproducible

```
def limpiar_reservas(tabla):
    t = tabla.copy()      # el original no se toca
    resumen = {"filas_antes": len(t)}
    ...                   # tus seis pasos
    return t, resumen
limpio, resumen = limpiar_reservas(crudo)
```

resumen (6 filas de ejemplo)	valor
filas_antes	6
duplicados_quitados	1
imposibles_a_nulo	2
antelaciones_recalculadas	1
filas_despues	5

Pasos, en este orden: **duplicados** → **precio** → **fechas** → **categorías** → **imposibles** → **antelación**.

Reproducible: misma entrada, misma salida. Nada de arreglar el CSV a mano.

HOY HAS APRENDIDO

Medir, decidir, comprobar

Diagnosticar

resumen_calidad, el original intacto y una copia por paso

Corregir

Duplicados, tipos, fechas por formato, categorías y antelación

Distinguir

Imposible: a nulo. Raro: se investiga. Nulo: pregunta por qué falta

Siguiente: S06_03 · preprocesado y pipelines